

1. Accuracy Score

Definition: Percentage of correct predictions made by the model.

Formula:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Where:

- TP = True Positive
- TN = True Negative
- FP = False Positive
- FN = False Negative

Example

Suppose a model predicts whether an email is spam.

Actual	Predicted
Spam	Spam
Not Spam	Not Spam
Spam	Spam
Spam	Not Spam

Correct Predictions = 3

Total Predictions = 4

Accuracy = $3/4 = 75\%$

Python Code

```
from sklearn.metrics import accuracy_score

y_true = [1,0,1,1]
y_pred = [1,0,1,0]

print(accuracy_score(y_true, y_pred))
```

Output:

0.75

2. Precision Score

Definition: Out of all positive predictions, how many were actually positive?

Formula:

$$Precision = \frac{TP}{TP+FP}$$

Real-Life Example

Cancer Detection:

- Model predicts 10 people have cancer.
- Actually only 8 have cancer.

Precision = $8/10 = 80\%$

High precision means fewer false alarms.

Python Code

```
from sklearn.metrics import precision_score

y_true = [1,0,1,1]
y_pred = [1,0,1,0]

print(precision_score(y_true, y_pred))
```

Output:

1.0

3. Recall Score

Definition: Out of all actual positive cases, how many did the model find?

Formula:

$$Recall = \frac{TP}{TP+FN}$$

Real-Life Example

Suppose:

- 100 patients actually have cancer.
- Model detects 90.

Recall = $90/100 = 90\%$

High recall means fewer missed cases.

Python Code

```
from sklearn.metrics import recall_score

y_true = [1,0,1,1]
y_pred = [1,0,1,0]

print(recall_score(y_true, y_pred))
```

Output:

0.67

4. F1 Score

Definition: Harmonic mean of Precision and Recall.

Formula:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Why use it?

When classes are imbalanced.

Example:

- Fraud Detection
- Disease Prediction
- Spam Detection

Python Code

```
from sklearn.metrics import f1_score

y_true = [1,0,1,1]
y_pred = [1,0,1,0]

print(f1_score(y_true, y_pred))
```

Output:

0.80

5. Confusion Matrix

A table showing prediction results.

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Example

```
from sklearn.metrics import confusion_matrix

y_true = [1,0,1,1]
y_pred = [1,0,1,0]

print(confusion_matrix(y_true, y_pred))
```

Output:

```
[[1 0]
 [1 2]]
```

Meaning:

```
TN = 1
FP = 0
FN = 1
TP = 2
```

Complete Example

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)

y_true = [1,0,1,1,0,1,0]
y_pred = [1,0,1,0,0,1,1]

print("Accuracy :", accuracy_score(y_true,y_pred))
print("Precision:", precision_score(y_true,y_pred))
print("Recall    :", recall_score(y_true,y_pred))
print("F1 Score :", f1_score(y_true,y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_true,y_pred))
```

Output:

```
Accuracy : 0.714
Precision: 0.75
Recall    : 0.75
F1 Score  : 0.75
```

Confusion Matrix:

```
[[2 1]  
 [1 3]]
```

- **Accuracy** → Overall correctness.
- **Precision** → "When model says YES, how often is it correct?"
- **Recall** → "How many actual YES cases did the model find?"
- **F1 Score** → Balance between Precision and Recall.
- **Confusion Matrix** → Complete picture of model performance.